Supplementary material

Miguel Luna García[1[0009−0006−6611−4360]], Iván Castilla Rodríguez[2,3[0000−0003−3933−2582]], Gara Miranda[3[0000−0002−2181−0067]], and Eduardo Segredo[3[0000−0002−4085−4356]]

Departamento de Ingeniería Informática y de Sistemas. Universidad de La Laguna,
Camino San Francisco de Paula, 19. Ap. 456.
38200, San Cristóbal de La Laguna, Spain
{miguel.luna.19, icasrod, gmiranda, esegredo}@ull.edu.es

A Selection of metaheuristics

Table 1: Summary of the selected metaheuristics

Metaheuristic	**Brief Description**
Iterated Local Search (ILS)	Finds new solutions through random perturbations and local searches.
Variable Neighbourhood Search (VNS)	Systematically explores different neighborhood structures to escape local optima.
Tabu Search (TS)	Uses memory to avoid cycles and escape local optima through temporarily forbidden moves.
Simulated Annealing (SA)	Simulates the cooling process of metals to accept worse solutions with a decreasing probability.
Greedy Randomized Adaptive Search Procedure (GRASP)	A randomized constructive algorithm followed by a local search.
Ant Colony Optimization (ACO)	Inspired by the behavior of ants, uses pheromone trails to construct new solutions.
Particle Swarm Optimization (PSO)	Based on the collective movement of particles through the solution space.
Evolutionary Algorithm (EA)	Uses evolutionary operators such as selection, crossover, and mutation to find new solutions.
Non-dominated Sorting Genetic Algorithm II (NSGA-II)	An evolutionary algorithm for multi-objective optimization, based on the dominance of solutions within Pareto fronts.
Strength Pareto Evolutionary Algorithm 2 (SPEA2)	Another multi-objective optimization technique that uses an external population and refined fitness measures.

B Metaheuristic examples

The representation of each one of these algorithms in the ontology can be found in the *individuals* folder in the repository[1].

B.1 Iterated Local Search

Iterated Local Search (ILS) [1] is a simple metaheuristic that uses a local search to improve solutions, and avoids premature convergence to a local optimum with the use of a random perturbation.

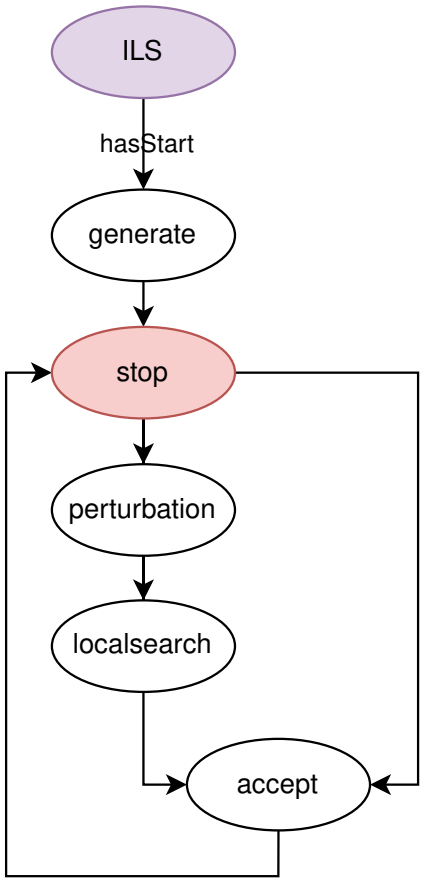

Fig. 1: Diagram of the ILS algorithm

[1] Link to the repository: https://github.com/ontologies-ULL/CO-META

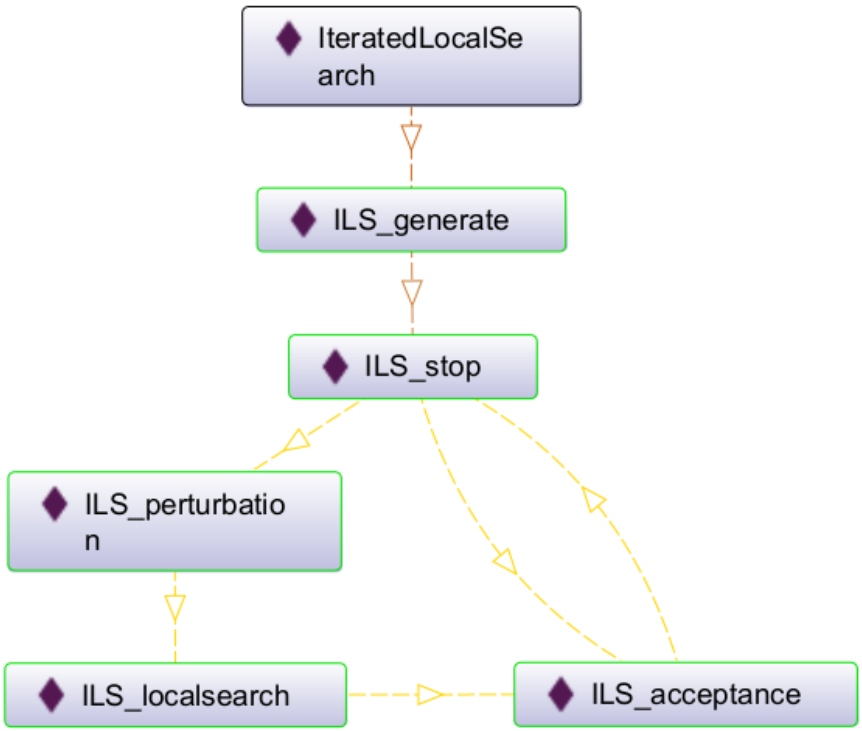

Fig. 2: OntoGraf representation of the ILS algorithm

B.2 Variable Neighbourhood Search

Variable Neighbourhood Search (VNS) [1] follows a similar principle to ILS, with the peculiarity that the neighborhood structure is variable. When a local optimum is reached, the neighborhood structure used in the perturbation is modified, changing the range of the random jump.

As shown in Figure 3, a new component, *change_neighbourhood*, is added to the algorithm. This component introduces the logic for modifying the neighborhood after the acceptance component. It compares the solution selected by the *accept* component with the solution used to start the iteration. If both are equal, it means that no better solution has been found and a local optimum has been reached, so the neighborhood is updated. Otherwise, the original structure is restored.

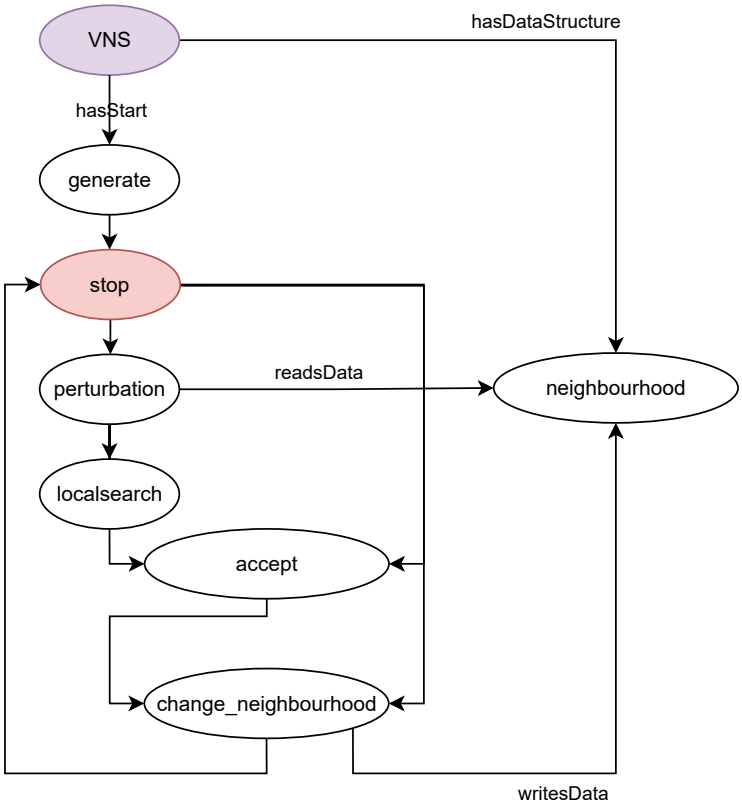

Fig. 3: Diagram of the VNS algorithm

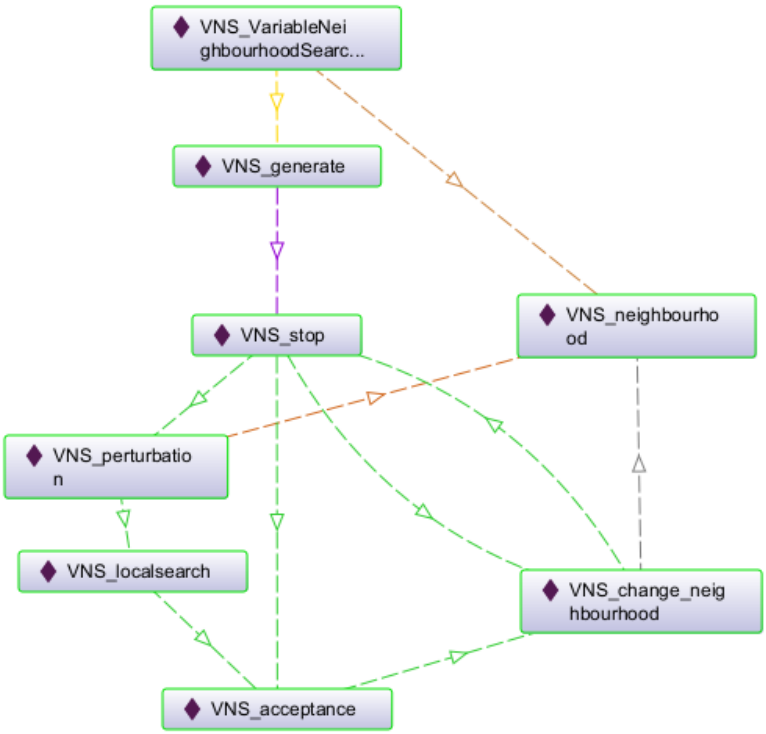

Fig. 4: OntoGraf representation of the VNS algorithm

B.3 Tabu Search

Tabu Search (TS) algorithm [1] stores previously visited solutions. In each iteration, the neighborhood of the current solution is generated, and a non-visited solution is selected.

As shown in Figure 5, the algorithm includes an *archive* component, which stores the visited solutions. The solutions contained in this list are removed from the neighborhood, and the best admissible solution is then selected to continue the search.

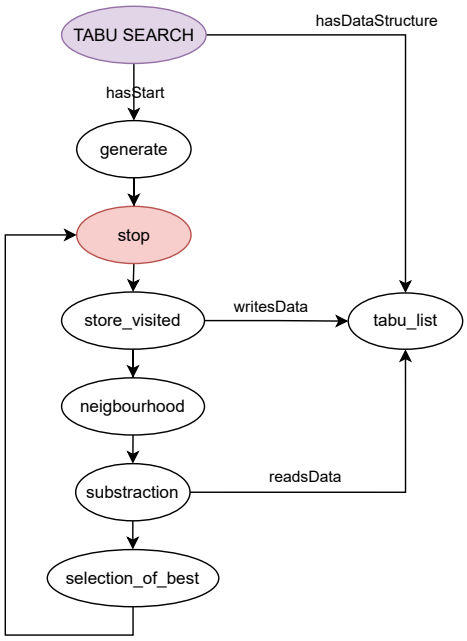

Fig. 5: Diagram of the TS algorithm

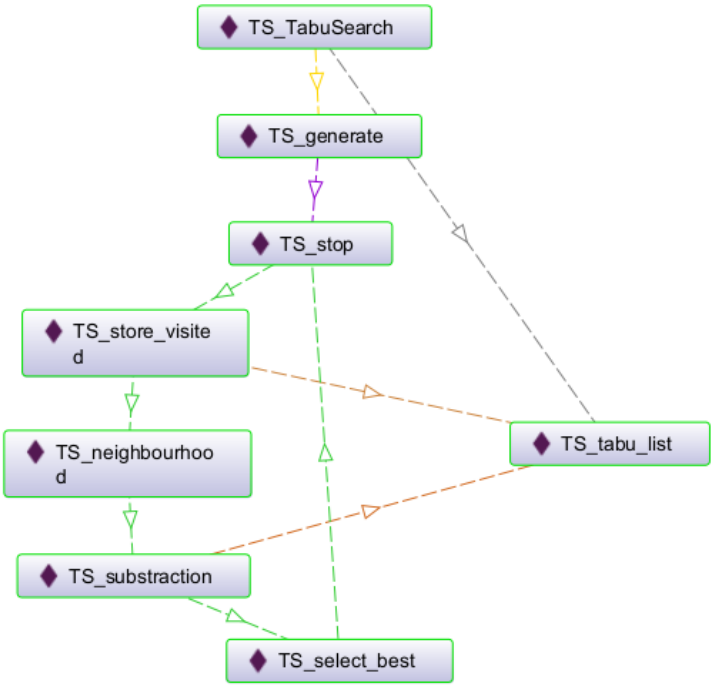

Fig. 6: OntoGraf representation of the TS algorithm

B.4 Simulated Annealing

Simulated Annealing (SA) [1] is based on a temperature parameter that decreases as the iterations progress. If the solution found after a perturbation is better than the current one, it replaces it. Otherwise, a worse solution may still be accepted with a probability that depends on the current temperature value. At high temperatures, the acceptance probability is higher, allowing uphill moves to escape local optima.

Figure 7 shows the main components of the algorithm, including the temperature based acceptance mechanism and the temperature reduction scheme.

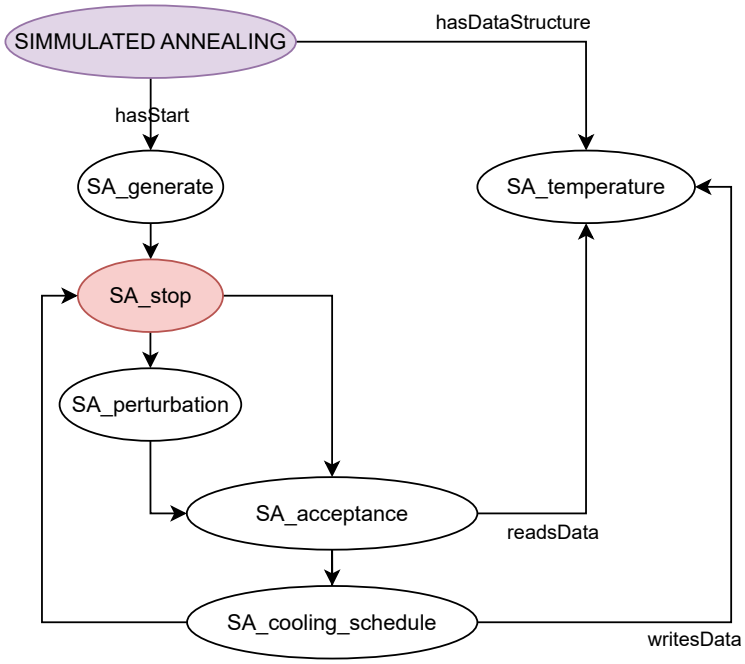

Fig. 7: Diagram of the SA algorithm

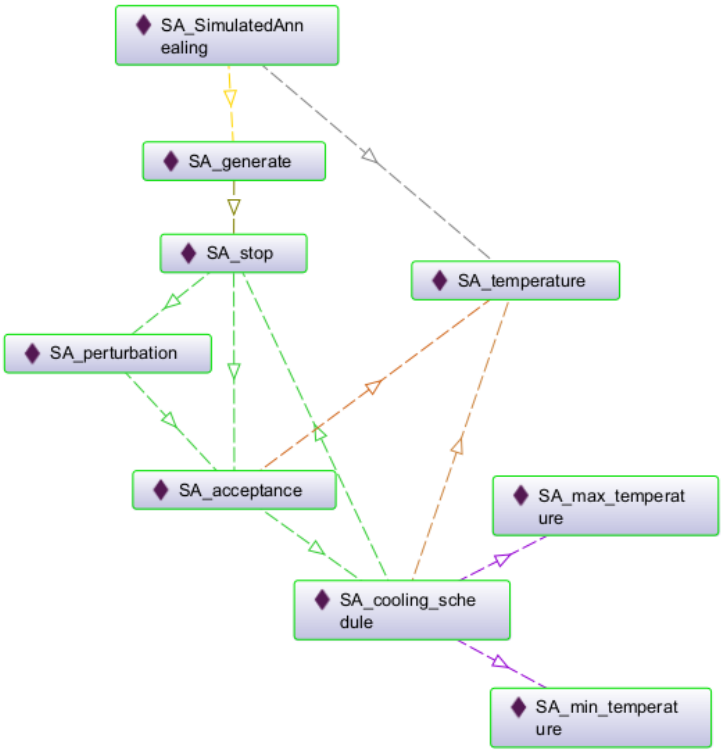

Fig. 8: OntoGraf representation of the SA algorithm

B.5 Greedy Randomized Adaptive Search Procedure

Greedy Randomized Adaptive Search Procedure (GRASP) [3] is a constructive metaheuristic in which solutions are iteratively built using a restricted candidate list that selects promising elements to be included in the solution until it is complete. After the construction phase, a local search is applied to improve the generated solution. The process is repeated multiple times, keeping track of the best solution found so far.

Figure 9 illustrates the component responsible for the solution construction phase.

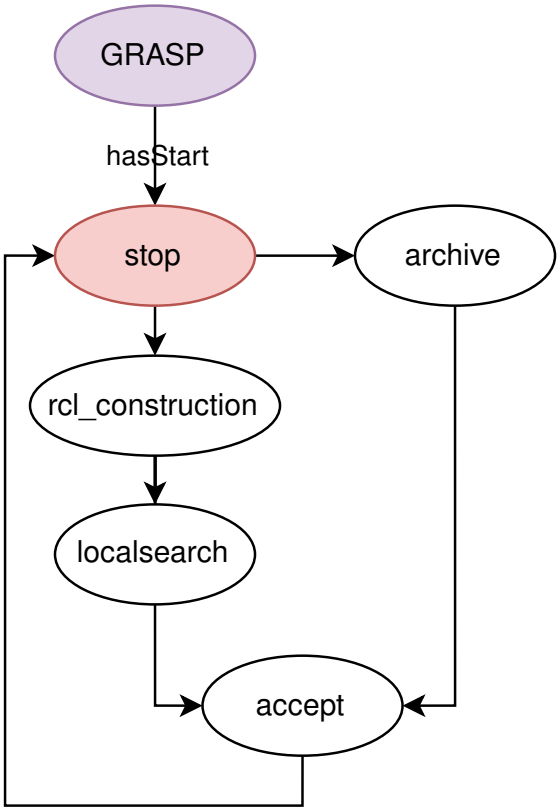

Fig. 9: Diagram of the GRASP algorithm

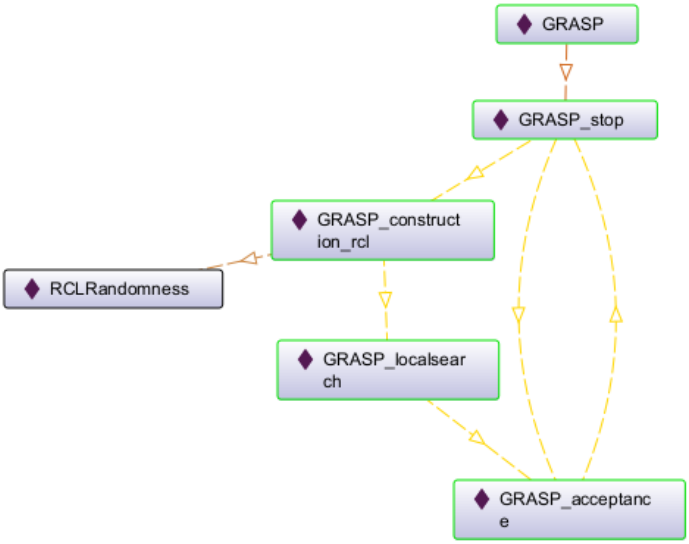

Fig. 10: OntoGraf representation of the GRASP algorithm

B.6 Ant Colony Optimization

Ant Colony Optimization (ACO) [1] is based on the construction of solutions guided by pheromones, a data structure that represents the frequency with which certain paths are chosen. Solutions are built according to the current pheromone levels, and after each iteration, the pheromone values are updated based on the objective function value of each constructed solution.

Figure 11 shows the components involved in the construction of the solution and pheromone updating, as well as an iteration component that manages the population.

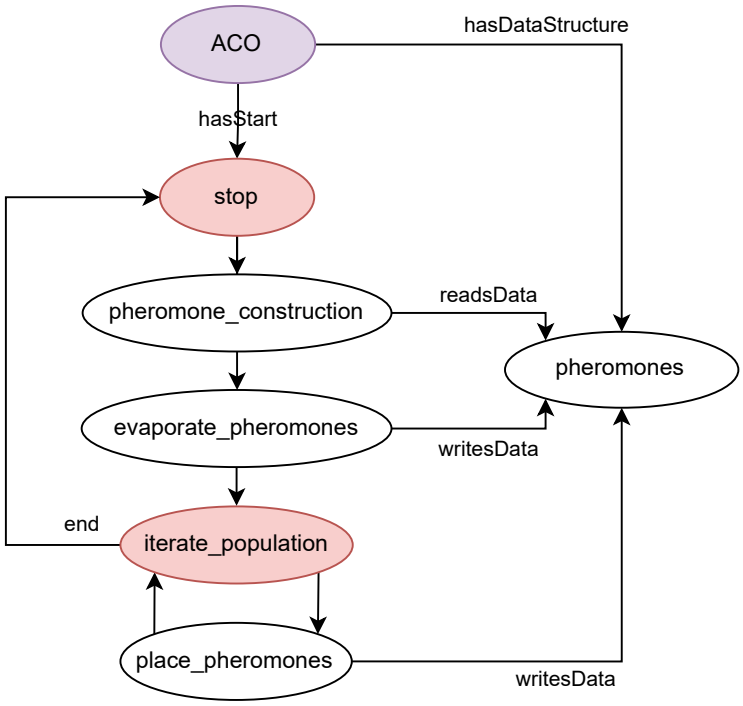

Fig. 11: Diagram of the ACO algorithm

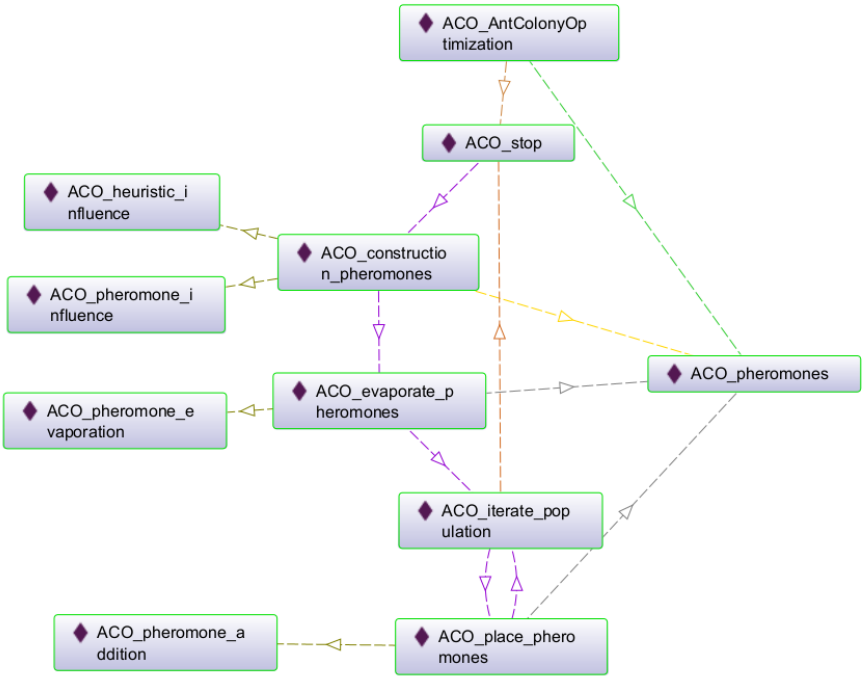

Fig. 12: OntoGraf representation of the ACO algorithm

B.7 Particle Swarm Optimization

Particle Swarm Optimization (PSO) [1] operates on a population of candidate solutions, called *particles*, which move through the solution space with a certain inertia. Its operation is illustrated in Figure 13.

An initial random set of particles is generated, and for each one, the best solution found so far (*personal best*) is stored, along with the best global solution (*global best*). Each particle is associated with a velocity, defined as a set of transformations applied to the particle's position. This velocity changes according to both the particle's personal best and the global best.

During each iteration, all particles are updated by recalculating their velocity, applying the corresponding transformations, and then updating the personal and global best values accordingly.

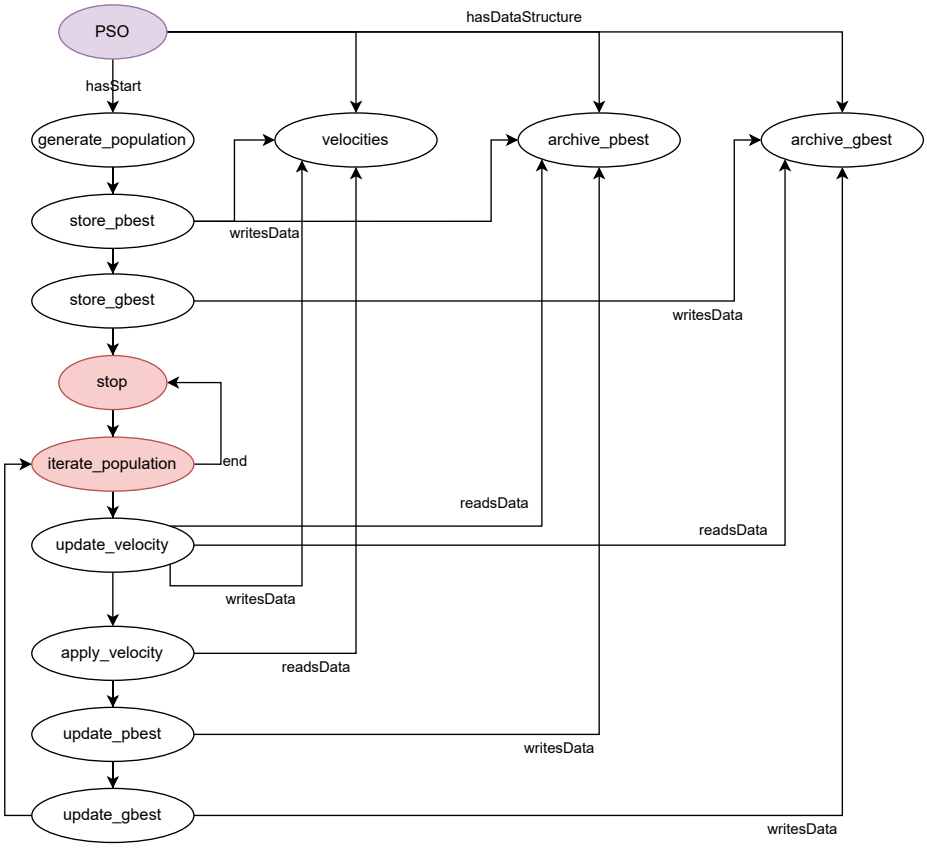

Fig. 13: Diagram of the PSO algorithm

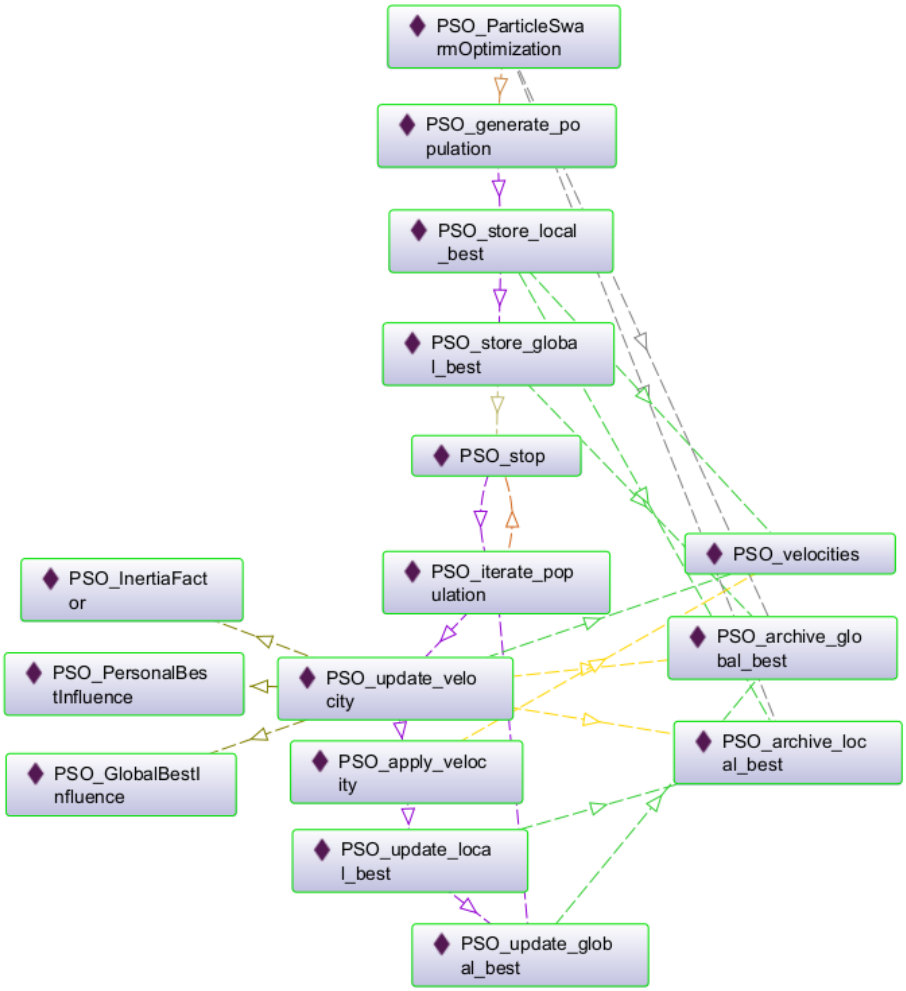

Fig. 14: OntoGraf representation of the PSO algorithm

B.8 Evolutionary Algorithms

Evolutionary Algorithms (EAs) [1] constitute a broad family of metaheuristics suitable for both single-objective and multi-objective optimization problems. They are inspired by the principles of biological evolution. As illustrated in Figure 15, each generation applies a set of evolutionary operators:

- *Selection:* selects, with replacement, the most promising solutions from the current generation to form the next one.
- *Crossover:* combines elements from pairs of solutions to generate new offspring solutions.
- *Mutation:* introduces random modifications to solutions to maintain diversity in the population.

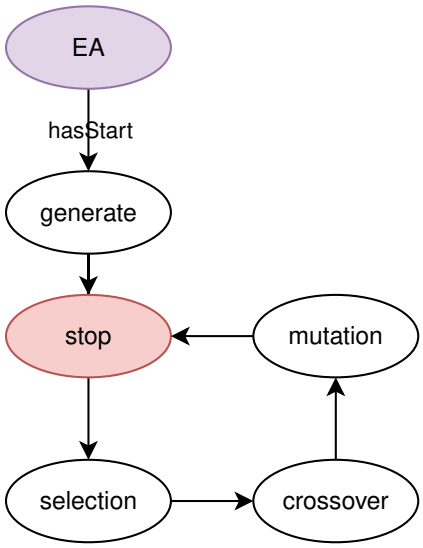

Fig. 15: Diagram of an Evolutionary Algorithm

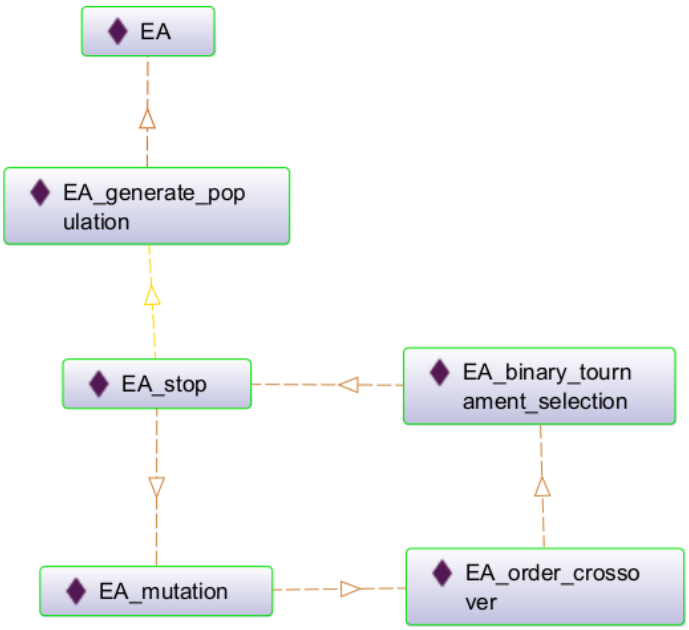

Fig. 16: OntoGraf representation of the EA algorithm

B.9 Non-dominated Sorting Genetic Algorithm II

NSGA-II [2] is a type of Evolutionary Algorithm designed to solve multi-objective optimization problems. Before applying operators that require a fitness value (such as the selection component), each solution must be assigned a unique fitness value, since initially multiple objectives exist. These fitness values are computed based on dominance relations among solutions and their relative distances in the objective space. Different approaches for fitness assignment and diversity preservation are discussed in [4].

As shown in Figure 17, the algorithm begins by applying fitness assignment operators to the initial population. In each iteration, the previous generation is merged with a new one obtained through selection, crossover, and mutation. The fitness assignment operators are then reapplied to the combined population, and the best solutions are selected until the original population size is restored.

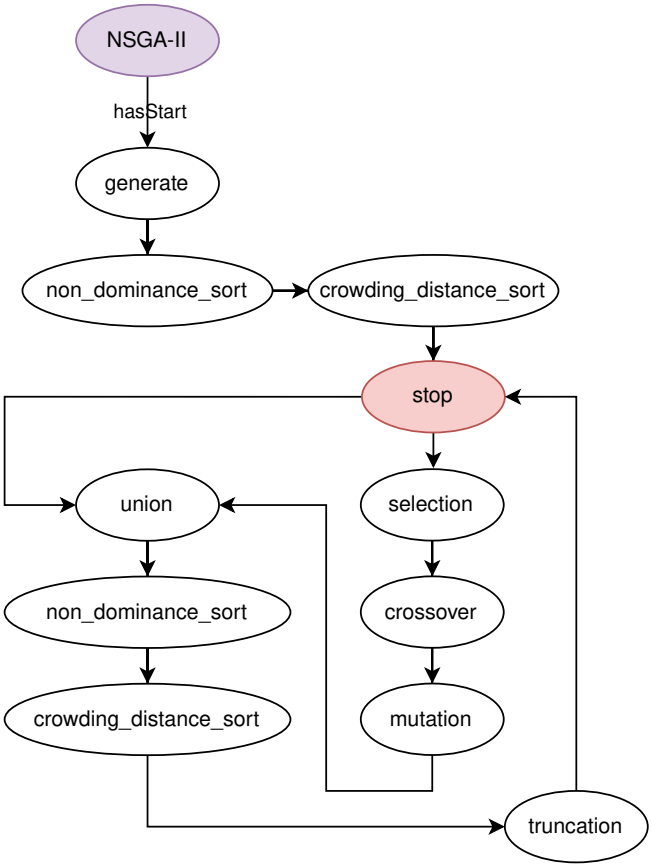

Fig. 17: Diagram of the NSGA-II algorithm

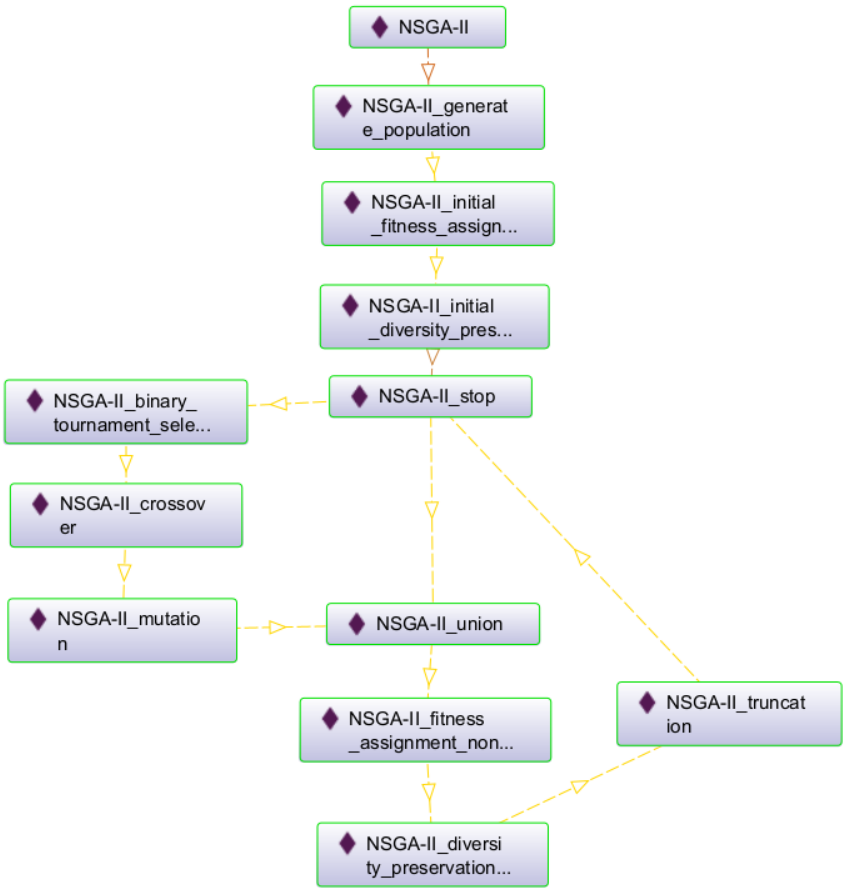

Fig. 18: OntoGraf representation of the NSGA-II algorithm

B.10 Strength Pareto Evolutionary Algorithm 2

SPEA2 [5] is another type of Evolutionary Algorithm that addresses multi-objective optimization problems. It follows principles similar to those of NSGA-II, but differs in its fitness assignment and diversity preservation mechanisms. As shown in Figure 19, the algorithm maintains an external archive that stores the best solutions found so far.

In each generation, the current population is combined with the archive, and the best individuals are selected to refill the archive based on their fitness and density information. The next generation is then created from the archive using the standard evolutionary operators: selection, crossover, and mutation.

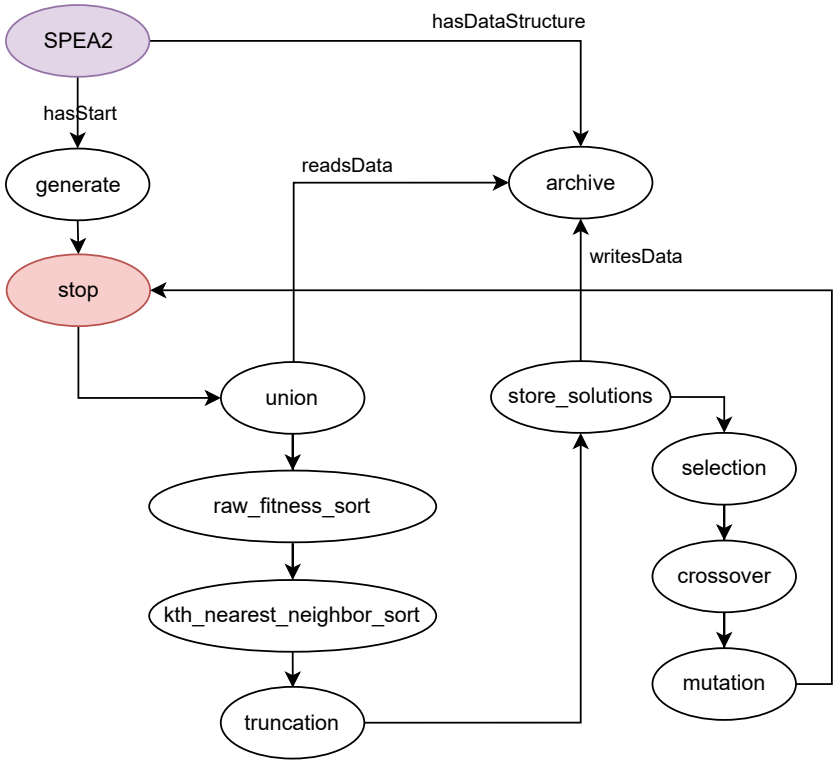

Fig. 19: Diagram of the SPEA2 algorithm

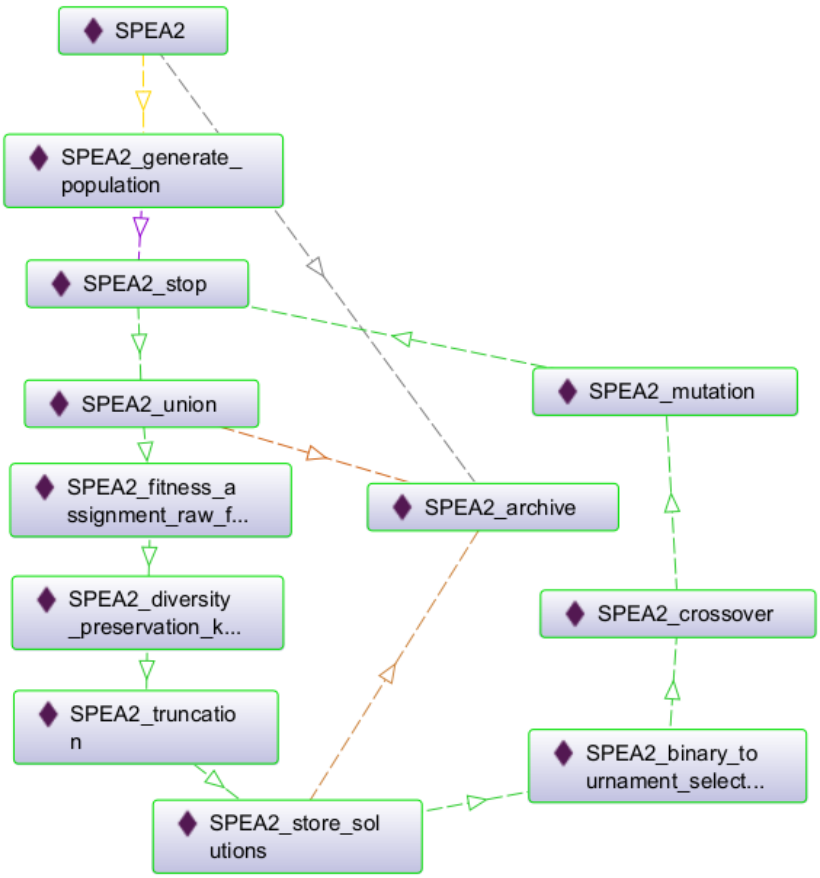

Fig. 20: OntoGraf representation of the SPEA2 algorithm

C SPARQL examples

The source codes for the following SPARQL queries can be found in the *supplementary/sparql* folder in the repository[2].

C.1 Metaheuristic Classification.

Figure 21 shows a query used to display the classes to which each metaheuristic belongs.

SPARQL query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX mh: <https://w3id.org/ontologies-ULL/CO-META#>
SELECT DISTINCT ?metaheuristic ?class
WHERE {
   ?metaheuristic a mh:Metaheuristic .
   ?metaheuristic a ?class .
   FILTER (
      ?class != mh:Metaheuristic &&
      ?class != owl:Class &&
      ?class != owl:NamedIndividual
   )
}
ORDER BY ?metaheuristic ?class
```

metaheuristic	class
AntColonyOptimization	ConstructiveMetaheuristic
AntColonyOptimization	PopulationBasedMetaheuristic
EvolutionaryAlgorithm	EvolutionaryAlgorithm
EvolutionaryAlgorithm	LocalSearchMetaheuristic
EvolutionaryAlgorithm	PopulationBasedMetaheuristic
GRASP	ConstructiveMetaheuristic
GRASP	SingleSolutionMetaheuristic

Fig. 21: SPARQL query showing the classes to which each algorithm belongs

C.2 List of Components.

Figure 22 shows a query for obtaining the list of components of an algorithm, and the class to which each component belongs. The query can be easily adapted for different algorithms by modifying the `VALUES` field.

C.3 List of Parameters.

Figure 23 shows a query for obtaining the list of parameters of an algorithm.

C.4 Similarities Between Metaheuristics.

Figure 24 shows a query that lists the common component types shared by two given metaheuristics, in this case Tabu Search and Simulated Annealing.

[2] Link to the repository: https://github.com/ontologies-ULL/CO-META

SPARQL query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX mh: <https://w3id.org/ontologies-ULL/CO-META#>
SELECT DISTINCT ?metaheuristic ?componentClass
WHERE {
  VALUES ?metaheuristic { mh:TabuSearch mh:SimulatedAnnealing }
  ?metaheuristic mh:isConnectedTo ?comp1 .
  ?comp1 a ?componentClass .

  ?componentClass rdfs:subClassOf+ mh:MetaheuristicComponent .

  FILTER NOT EXISTS {
    ?subclass rdfs:subClassOf ?componentClass .

    ?mh1 mh:isConnectedTo ?compSub1 .
    ?compSub1 a ?subclass .
  }
  FILTER (
    ?componentClass != owl:NamedIndividual &&
```

metaheuristic	componentClass
SimulatedAnnealing	StoppingCriterionComponent
SimulatedAnnealing	ResourceHandlerComponent
SimulatedAnnealing	SingleSolutionGenerationComponent
SimulatedAnnealing	MutationComponent
SimulatedAnnealing	AcceptanceComponent
TabuSearch	StorageComponent

Fig. 22: SPARQL query retrieving the list of components of an algorithm

SPARQL query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX mh: <https://w3id.org/ontologies-ULL/CO-META#>
SELECT DISTINCT ?metaheuristic ?component ?parameter
WHERE {
        ?metaheuristic a mh:Metaheuristic .
                ?metaheuristic mh:isConnectedTo ?component .
        ?component a mh:MetaheuristicComponent .
                ?component mh:hasParameter ?parameter .
}
ORDER BY ?metaheuristic
```

metaheuristic	component	parameter
AntColonyOptimization	ACO_construction_pheromones	PopulationSize
AntColonyOptimization	ACO_stop	Iterations
EvolutionaryAlgorithm	EA_mutation	MutationProbability
EvolutionaryAlgorithm	EA_order_crossover	CrossoverProbability
EvolutionaryAlgorithm	EA_stop	Iterations
EvolutionaryAlgorithm	EA_generate_population	PopulationSize

Fig. 23: SPARQL query retrieving the list of parameters of an algorithm

SPARQL query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX mh: <https://w3id.org/ontologies-ULL/CO-META#>
SELECT DISTINCT ?componentClass
WHERE {
  VALUES (?mh1 ?mh2) {
    (mh:TabuSearch mh:SimulatedAnnealing)
  }
  ?mh1 mh:isConnectedTo ?comp1 .
  ?comp1 a ?componentClass .
  ?mh2 mh:isConnectedTo ?comp2 .
  ?comp2 a ?componentClass .
  ?componentClass rdfs:subClassOf+ mh:MetaheuristicComponent .
  FILTER NOT EXISTS {
    ?subclass rdfs:subClassOf ?componentClass .
    ?mh1 mh:isConnectedTo ?compSub1 .
    ?compSub1 a ?subclass .
    ?mh2 mh:isConnectedTo ?compSub2 .
    ?compSub2 a ?subclass
```

componentClass
AcceptanceComponent
ModificationComponent
SingleSolutionGenerationComponent
StoppingCriterionComponent

Fig. 24: SPARQL query listing the common components between two metaheuristics

References

1. Chopard, B., Tomassini, M.: An Introduction to Metaheuristics for Optimization. Springer International Publishing (2018). https://doi.org/10.1007/978-3-319-93073-2
2. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: Nsga-ii. IEEE Transactions on Evolutionary Computation **6**(2), 182–197 (2002). https://doi.org/10.1109/4235.996017
3. Feo, T.A., Resende, M.G.C.: Greedy randomized adaptive search procedures. Journal of Global Optimization **6**(2), 109–133 (Mar 1995). https://doi.org/10.1007/BF01096763
4. Talbi, E.G.: A unified view of metaheuristics for multi-objective optimization (2016), https://www.youtube.com/watch?v=ITNHQNAy7AQ, Accessed: March 12, 2026
5. Zitzler, E., Laumanns, M., Thiele, L.: Spea2: Improving the strength pareto evolutionary algorithm. TIK Report 103 (2001), https://api.semanticscholar.org/CorpusID:16584254